

ENTERPRISE VALUE REALIZATION THROUGH PREDICTIVE MAINTENANCE

A Case Study of a Live, Deployed AI Platform

LIVE DEPLOYMENT

Platform: SmartFactory — Industrial IoT / Predictive Maintenance Dashboard

Live URL: <https://pdm-dashboard.netlify.app/>

Status: Deployed and reachable; screenshots in Section 6 are captured directly from this build.

Demo credentials are available on request and are intentionally omitted from this document.

Project: AI Predictive Maintenance Platform for Manufacturing

Target Audience: Manufacturing C-Suite, Plant Directors, Enterprise Architects, Investors

Author: Ujjwal Dwivedi

1. Executive Summary

Manufacturing has reached an inflection point where legacy maintenance strategies — both reactive and strictly calendar-based preventive — are increasingly costly. Industry estimates commonly cited across the sector put unplanned downtime in the tens of billions of dollars annually across global manufacturing.

This case study documents a working AI Predictive Maintenance platform that has been designed, built, and deployed as a live prototype, reachable today at pdm-dashboard.netlify.app. The platform combines time-series machine learning (LSTM, XGBoost, Isolation Forest) with a modern microservices architecture (FastAPI, React, PostgreSQL, InfluxDB) to move maintenance decisions from reactive firefighting or fixed schedules toward continuous, data-driven monitoring.

What follows is a transparent walkthrough: the problem the platform targets, how it is built, what the live dashboard actually shows today (Section 6, with real screenshots), and — separately and clearly labeled — an industry-benchmark model of what a factory could realistically expect once the platform is connected to real sensor data (Section 8). The two are kept distinct throughout this document: demonstrated platform behavior is never blended with projected financial impact.

ABOUT THIS DOCUMENT

This version replaces an earlier draft that used hypothetical/reverse-engineered narrative framing.

Every screenshot below is taken directly from the live deployment.

Every financial figure is labeled as either (a) live output from the current demo dataset, or (b) an illustrative, industry-benchmark projection — never presented as an audited or measured production result.

2. Industry Background & Problem Statement

The Cost of Ignorance in Manufacturing

Historically, industrial maintenance falls into two paradigms:

- **Reactive (Run-to-Failure):** Results in catastrophic breakdowns, safety hazards, cascading equipment damage, and costly expedited shipping for spare parts.
- **Preventive (Time-Based):** Results in over-maintenance. Functional parts are replaced prematurely, inflating spare inventory and labor costs, while still failing to catch a large share of random equipment failures.

The Problem: manufacturing companies often lack a unified, accessible, and accurate system to monitor asset health in real time and anticipate failures before they occur.

Target Industries & Applicability

The platform's architecture is domain-agnostic. Illustrative applicability includes:

- **Automotive & Heavy Machinery:** robotic-arm servo failures, CNC spindle degradation.
- **Power Plants & Oil/Gas:** turbines, compressors, pump cavitation.

- Pharmaceuticals & Food Processing: environmental control (HVAC/chiller health) to prevent batch spoilage.
- Mining, Cement & Steel: conveyor belt tears, crusher motor bearing failures, furnace refractory wear.

3. Proposed Solution & Technical Stack

The platform ingests sensor telemetry, processes it through machine learning models, and surfaces actionable intelligence through the centralized dashboard shown in Section 6.

Core Technology Choices

- Backend (Python/FastAPI): asynchronous request handling suited to high-throughput IoT endpoints.
- Database (PostgreSQL + InfluxDB): a dual-database strategy — PostgreSQL for relational data (users, RBAC, asset metadata), InfluxDB for high-velocity sensor time series.
- Frontend (React/TypeScript): a type-safe, component-driven UI, deployed live on Netlify.
- AI Engine (LSTM, XGBoost, Isolation Forest): a hybrid approach — LSTM for temporal degradation patterns, Isolation Forest for anomaly detection, XGBoost for failure-mode classification.
- Infrastructure (Docker, Netlify, Cloud): containerization for environment parity across local, testing, and production.

4. Technical Architecture

The system is decoupled into logical tiers to support high availability and fault tolerance:

ARCHITECTURE OVERVIEW

EDGE / IoT — Vibration, Temperature, Power sensors → IoT Gateway (MQTT/HTTP)

CLOUD — FastAPI Gateway → Data Tier (PostgreSQL & InfluxDB) → AI Engine (LSTM / XGBoost / Isolation Forest)

PRESENTATION — React/TypeScript dashboard, deployed at `pdm-dashboard.netlify.app`

5. AI Models & The Data Pipeline

The Dataset: NASA C-MAPSS Turbofan Engine

The platform's baseline models were trained on the industry-standard NASA Commercial Modular Aero-Propulsion System Simulation (C-MAPSS) dataset.

- Why it's standard: multi-variate time-series data reflecting run-to-failure degradation under varying operating conditions.
- Limitations: it is synthetic and stationary. Real factory data carries noise, sensor drift, and non-linear operational state changes — which is why Section 10 lists real-data fine-tuning as an open item rather than a solved problem.

- Feature engineering: MinMax normalization, moving averages to smooth noise, and sliding windows (e.g., 30 cycles) to prepare sequences for the LSTM network.

Sensor Data: The Lifeblood of PdM

- Vibration: often the earliest indicator of bearing wear or misalignment.
- Temperature & Pressure: indicators of friction, coolant failure, or blockages.
- Power Consumption (current/voltage): spikes can indicate mechanical resistance or electrical degradation.
- RPM: correlated with other metrics to normalize operational load.

AI Prediction Pipeline

PIPELINE

Raw Sensor Data → Data Cleaning & Imputation → Feature Engineering

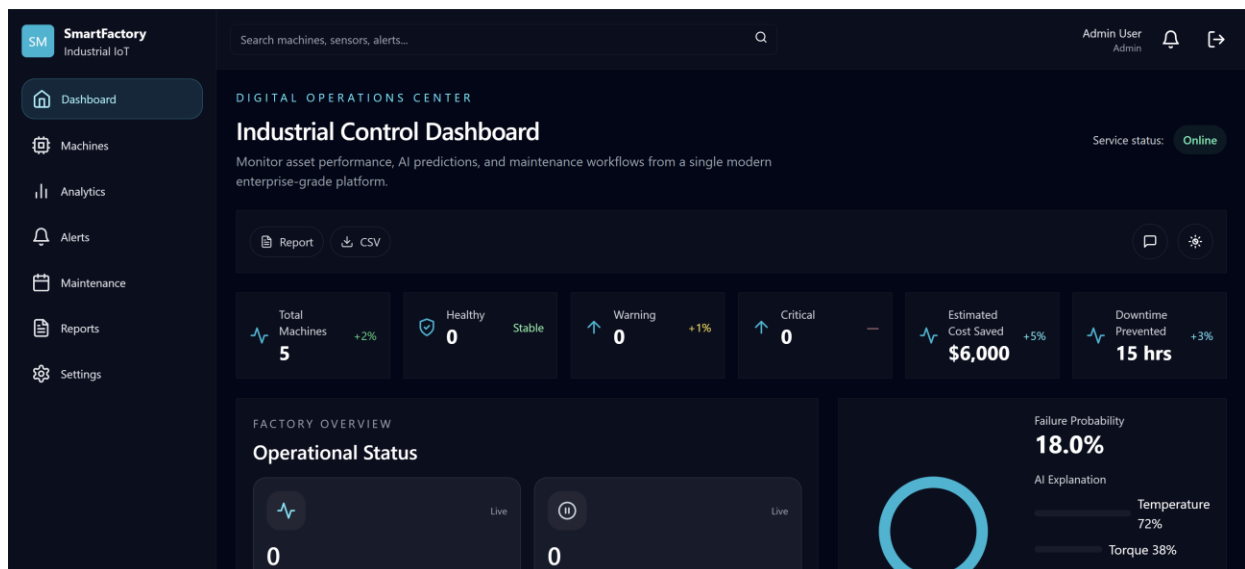
Model Routing: LSTM (sequential) · XGBoost (tabular) · Isolation Forest (unsupervised)

Prediction Aggregator → Output: Failure Probability & Remaining Useful Life (RUL)

6. Platform Overview — Live Walkthrough

The screenshots below are captured directly from the live deployment at pdm-dashboard.netlify.app, using the platform's current demo dataset (5 monitored assets).

6.1 Executive Dashboard

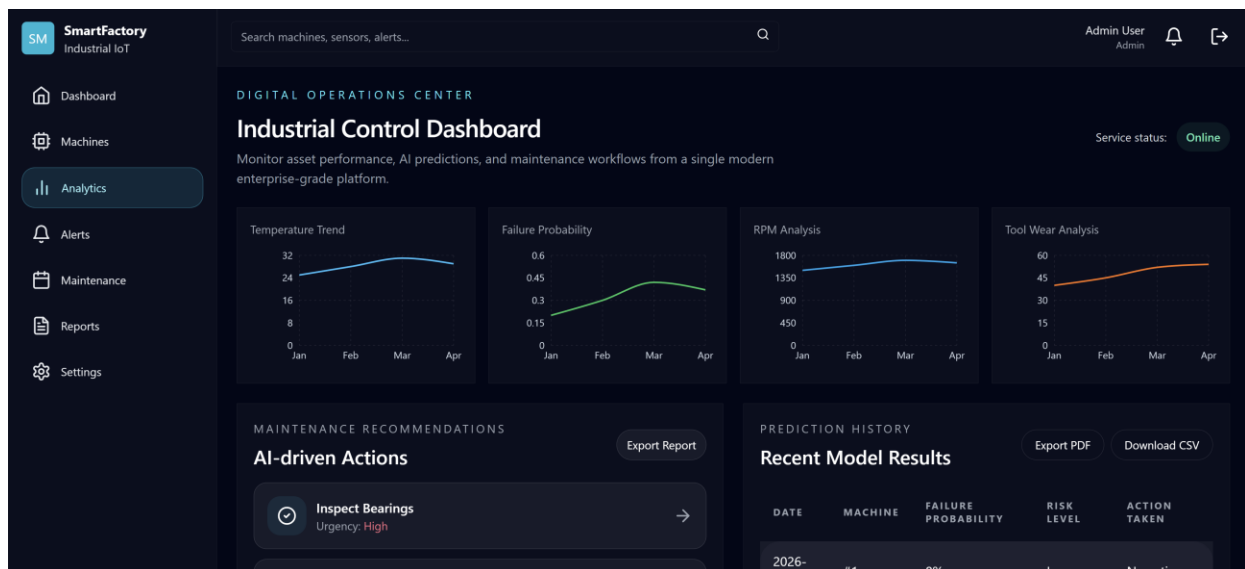


Live screenshot — Industrial Control Dashboard, captured July 2, 2026.

The dashboard surfaces fleet-level KPIs at a glance: Total Machines (5), a Healthy / Warning / Critical breakdown, Estimated Cost Saved, and Downtime Prevented. A Failure Probability gauge (18.0% at the time of

capture) is paired with an AI Explanation panel that decomposes the prediction into contributing factors — Temperature and Torque are shown as the leading drivers in this instance.

6.2 Analytics



Live screenshot — Analytics view, captured July 2, 2026.

The Analytics view trends four signals over time — Temperature, Failure Probability, RPM, and Tool Wear — and pairs them with two operational feeds: AI-driven Maintenance Recommendations (e.g., "Inspect Bearings," flagged Urgency: High) and a Prediction History log recording each model run by date, machine, failure probability, risk level, and action taken.

6.3 Feature Summary

- Authentication & Security (JWT): Role-Based Access Control separates Admin, Plant Manager, and Technician views.
- Executive Dashboard: fleet health, active alerts, and cost/downtime estimates (Section 6.1).
- Machine Health Matrix: drill-down per-asset telemetry.
- Prediction & AI Insights: failure probability, RUL, and contributing-factor explanation (Section 6.1).
- Alerts & Maintenance Engine: rule-based and model-driven maintenance recommendations (Section 6.2).

7. Deployment Journey & Scalability

The build followed a phased approach:

- Phase 1–3 (Local to API): Python environment setup, EDA on the NASA dataset, model export, and FastAPI endpoints.
- Phase 4–5 (DB & Docker): relational schema and time-series bucket design; Dockerized backend and frontend.

- Phase 6–8 (Dashboard & Cloud): frontend build-out and deployment to Netlify — the live build referenced throughout this document.
- Phase 9 (Planned — IoT): provisioning MQTT brokers for real-time edge-device ingestion, the key remaining step to move from demo data to live factory feeds.

8. Business & Financial Impact

This section is deliberately split into two parts that should not be conflated: what the live platform currently shows on its demo dataset, and a separate, clearly hypothetical model of factory-level savings based on published industry benchmarks.

8.1 Live Platform Output (Current Demo Data)

Metric	Live Value (Demo Dataset)
Total Machines Monitored	5
Healthy / Warning / Critical	0 / 0 / 0
Estimated Cost Saved	\$6,000 (+5%)
Downtime Prevented	15 hrs (+3%)
Sample Failure Probability (flagged asset)	18.0%

These figures are read directly from the dashboard shown in Section 6.1. They demonstrate that the calculation pipeline and UI work end-to-end — from sensor input through model output to a plant-manager-facing number. They are not, however, validated savings from a production factory: the underlying dataset is a small demo set, not a live sensor feed from an operating plant.

8.2 Modeled Impact — Industry-Benchmark Projection

The table below is an illustrative model, not a measured result. It applies commonly cited industry benchmark ranges for predictive-maintenance programs to typical baseline metrics; actual results depend heavily on asset mix, data quality, and sensor coverage at a specific site.

Metric	Traditional Baseline	Benchmark PdM Range	Illustrative Net Change
Unplanned Downtime	~10% of operating hours	3–6% of operating hours	Meaningful reduction, site-dependent
Maintenance Labor	High (constant inspection)	Targeted, condition-based	Cost saving, magnitude varies

Metric	Traditional Baseline	Benchmark PdM Range	Illustrative Net Change
Spare Parts Inventory	High (just-in-case)	Leaner, just-in-time	Reduction, magnitude varies
Asset Lifespan	Baseline	Modest extension reported in industry studies	Extension, not yet measured here

8.3 Illustrative ROI Scenario (Hypothetical Factory)

The worked example below shows how the platform's ROI framing could be applied once connected to a real factory's downtime data. All inputs are hypothetical and provided only to illustrate the calculation — they are not this platform's own measured output.

- Hypothetical assumption: a plant loses 400 hours/year to unplanned downtime, valued at \$10,000/hour of lost production.
- Hypothetical annual downtime cost: \$4,000,000.
- If a mid-range benchmark reduction (illustratively, ~50–70%) were achieved: roughly \$2M–\$2.8M in avoided cost per year — again, hypothetical, and contingent on real-world validation.
- Against a representative cloud/software deployment cost of \$150,000/year, this hypothetical scenario implies a fast payback — but this has not been demonstrated on a real deployment and should be treated as a planning input, not a claim.

9. Competitive Analysis

The comparison below reflects design intent — target market, architectural philosophy, and cost model — rather than benchmarked performance testing against these vendors, which has not been conducted.

Feature / Vendor	This Platform	Siemens Senseye	GE Vernova (Predix)	AWS Lookout
Target Market	SMEs to mid-market	Large enterprise	Fortune 100	AWS ecosystem
Architecture	Open-source stack, flexible	Proprietary	Proprietary, heavy	Managed AWS service
Cost Model	Cloud-usage based	High (SaaS + setup)	Very high	Moderate/high
Customization	Full code control	Low/medium	Low	Medium

Independent, head-to-head benchmarking against these platforms — on accuracy, latency, or total cost of ownership — is a natural next step and is not yet part of this case study.

10. SWOT Analysis

Strengths

Modern, decoupled tech stack; live, working dashboard; low total cost of ownership relative to enterprise incumbents; fast iteration cycle.

Weaknesses

Currently runs on a small demo dataset (5 simulated assets), not yet integrated with a live plant's sensor feed or historical maintenance records. Underlying models are trained on the synthetic NASA C-MAPSS dataset and still require fine-tuning on real factory telemetry before failure-probability outputs can be trusted for operational decisions. No native sensor hardware offering. Section 8's financial figures are largely illustrative pending a real pilot.

Opportunities

A pilot deployment with a real manufacturing partner would convert the illustrative Section 8.2/8.3 figures into measured results. Longer-term: digital twins, a generative-AI maintenance copilot, and SaaS licensing for SMEs.

Threats

Deep-pocketed incumbents (Siemens, GE, AWS, IBM); variable sensor-data quality at legacy plants; organizational resistance to AI-driven maintenance decisions.

11. Technical Challenges & Engineering Realities

- Data Imbalance: factories run normally the vast majority of the time; failure events are rare. Approach: SMOTE-based oversampling and Isolation Forest / Autoencoder methods that only require "normal" operating data to flag anomalies.
- Latency vs. Batching: running a full LSTM inference on every high-frequency sensor ping is costly. Approach: feature aggregation and inference on sliding windows (e.g., 5-minute) rather than millisecond intervals.
- Sensor Noise: factory floors are electrically noisy. Approach: low-pass filters and moving averages applied before model input.

12. Future Roadmap: The Path to Autonomous Factories

- Phase A — Prescriptive Maintenance (Months 1–6): ERP/SAP integration for automated spare-parts procurement; energy-optimization modules.
- Phase B — Immersive & Edge Tech (Months 6–12): edge AI / federated learning on IoT gateways to cut cloud egress cost; AR-guided technician mobile app.
- Phase C — Generative AI Copilot (Months 12–18): LLM/RAG integration trained on OEM manuals so technicians can ask natural-language questions about predicted failures and isolation steps.

The single highest-priority near-term milestone, ahead of all three phases above, is connecting the platform to one real factory's live sensor feed — this is what would move Section 8 from a benchmark model to a measured case.

13. Conclusion & Key Takeaways

This platform is a live, working system — not a concept deck. The screenshots in Section 6 show an operating dashboard computing fleet health, cost-saving estimates, and failure probabilities end-to-end on its current demo dataset. That is a genuine, verifiable milestone.

What it is not yet, and what this document is careful not to imply, is a validated production deployment. The financial figures in Section 8 are honestly split between what the platform actually outputs today and what industry benchmarks suggest is achievable once it is connected to a real plant's sensor data.

Key Takeaways for Stakeholders

- For Plant Managers: a working UI for asset-health visibility exists today and can be evaluated hands-on at the live URL.
- For the CTO: a modern, decoupled architecture (FastAPI/React/PostgreSQL/InfluxDB) that is deployed, not just diagrammed.
- For Investors: a functioning MVP with a clear, honestly labeled path — a real-factory pilot — to convert benchmark projections into measured ROI.

LIVE DEMO ACCESS

URL: <https://pdm-dashboard.netlify.app/>

Login credentials are available on request and have intentionally been left out of this document.